getDerivedStateFromError
getIntensidad
getMetrics
getPreviewText
getProfile
getRegistroByFecha
getRegistroById
getRegistros
getRegistrosByRango
getStepLabel
goToNext
goToPrevious
handleAddActividad
handleAddContact
handleCall
handleCancelAdd
handleCancelEditor
handleChange
handleCreateEntry
handleDeleteEntry
handleEditEntry
handleEmocionToggle
handleImageError
handleInputChange
handleIntensidadChange
handleLogin
handleLogout
handleOverlayClick
handleRegister
handleReload
handleRemoveActividad
handleSendEmail
handleSendSms
handleShareEntry
handleSubmit
handleToggle
handleToggleExpand
handleUpdateEntry
healthCheckController
info
instrumentMongoDB
intensidad
isMobileDevice
isSelected
loadContacts
loadEntries
loading
logAuthAttempt
loginUser
logsDir
metricsMiddleware
nextStep
nombre
obtenerArticulos
obtenerArticulosDeMedicalAPIs
obtenerArticulosDeNewsAPI
obtenerArticulosDePubMed
obtenerArticulosLocales

Global

Members

404_Error_Handler

Description: Middleware para capturar errores 404 (Not Found).

Source: backend/src/app.js, line 94

Middleware para capturar errores 404 (Not Found).

(constant) ACTIVIDADES :Array.<string>

Description: Lista de actividades físicas predefinidas.

Source: frontend/src/components/molecules/ActivitySelector.jsx, line 14

Lista de actividades físicas predefinidas.

Type:

- `Array.<string>`

(constant) API_URL :string

Description: La URL base del backend, obtenida de las variables de entorno o un valor por defecto. Lee de la variable de entorno REACT_APP_API_URL. Fallback a http://localhost:4000 solo para desarrollo local. ⚠ IMPORTANTE: En producción, REACT_APP_API_URL DEBE estar configurada en las variables de entorno.

Source: frontend/src/config/api.js, line 6

La URL base del backend, obtenida de las variables de entorno o un valor por defecto. Lee de la variable de entorno REACT_APP_API_URL. Fallback a http://localhost:4000 solo para desarrollo local. ⚠ IMPORTANTE: En producción, REACT_APP_API_URL DEBE estar configurada en las variables de entorno.

Type:

- `string`

(constant) API_URL :string

Description: URL base de la API del backend. Lee de la variable de entorno REACT_APP_API_URL. Fallback a http://localhost:4000 solo para desarrollo local. En producción, REACT_APP_API_URL DEBE estar configurada.

Source: frontend/src/config/api.service.js, line 8

URL base de la API del backend. Lee de la variable de entorno REACT_APP_API_URL. Fallback a http://localhost:4000 solo para desarrollo local. En producción, REACT_APP_API_URL DEBE estar configurada.

Type:

- `string`

(constant) ASPECTOS_COGNITIVOS :Array.<string>

Description: Lista de aspectos cognitivos predefinidos.

Source: frontend/src/components/molecules/CognitionSelector.jsx, line 16

Lista de aspectos cognitivos predefinidos.

Type:

- `Array.<string>`

CORS_Configuration

Description: Configuración de CORS para permitir peticiones desde el frontend.

Source: backend/src/app.js, line 33

Properties:

`origin`	`string`	El origen permitido para las peticiones.
`credentials`	`boolean`	Indica si se permiten credenciales.
`optionsSuccessStatus`	`number`	Código de estado para peticiones OPTIONS.

Configuración de CORS para permitir peticiones desde el frontend.

(constant) EMOCIONES :Array.<string>

Description: Lista de emociones predefinidas.

Source: frontend/src/components/molecules/EmotionSelector.jsx, line 16

Lista de emociones predefinidas.

Type:

- `Array.<string>`

GET_/

Description: Ruta de prueba para comprobar que el servidor funciona.

Source: backend/src/app.js, line 83

Ruta de prueba para comprobar que el servidor funciona.

GET_/api/metrics

Description: Endpoint para obtener métricas en formato Prometheus

Source: backend/src/app.js, line 69

Endpoint para obtener métricas en formato Prometheus

Generic_Error_Handler

Description: Middleware para gestionar todos los demás errores.

Source: backend/src/app.js, line 106

Middleware para gestionar todos los demás errores.

(constant) api :AxiosInstance

Description: Instancia de Axios configurada para las peticiones a la API.

Source: frontend/src/config/api.service.js, line 17

Instancia de Axios configurada para las peticiones a la API.

Type:

- `AxiosInstance`

description

Description: La descripción de la tarjeta.

Source: frontend/src/components/molecules/Carousel.jsx, line 102

La descripción de la tarjeta.

duracion

Description: La duración de la actividad en minutos.

Source: frontend/src/components/molecules/ActivitySelector.jsx, line 144

La duración de la actividad en minutos.

emotion

Description: El nombre de la emoción.

Source: frontend/src/components/molecules/EmotionStats.jsx, line 57

El nombre de la emoción.

intensidad

Description: La intensidad de la actividad.

Source: frontend/src/components/molecules/ActivitySelector.jsx, line 146

La intensidad de la actividad.

intensidad

Description: La intensidad del aspecto cognitivo.

Source: frontend/src/components/molecules/CognitionSelector.jsx, line 114

La intensidad del aspecto cognitivo.

intensidad

Description: La intensidad de la emoción.

Source: frontend/src/components/molecules/EmotionSelector.jsx, line 127

La intensidad de la emoción.

(constant) logsDir

Description: Configuración de Winston Logger

> Source: backend/src/services/observability.service.js, line 68

Configuración de Winston Logger

nombre

> Description: El nombre de la actividad.
>
> Source: frontend/src/components/molecules/ActivitySelector.jsx, line 142

El nombre de la actividad.

nombre

> Description: El nombre del aspecto cognitivo.
>
> Source: frontend/src/components/molecules/CognitionSelector.jsx, line 112

El nombre del aspecto cognitivo.

nombre

> Description: El nombre de la emoción.
>
> Source: frontend/src/components/molecules/EmotionSelector.jsx, line 125

El nombre de la emoción.

percentage

> Description: El porcentaje de la emoción.
>
> Source: frontend/src/components/molecules/EmotionStats.jsx, line 59

El porcentaje de la emoción.

(constant) port :number

> Description: El puerto en el que se ejecutará el servidor.
>
> Source: backend/src/server.js, line 56

El puerto en el que se ejecutará el servidor.

Type:

- `number`

(constant) register

> Description: Configuración de Prometheus Metrics
>
> Source: backend/src/services/observability.service.js, line 17

Configuración de Prometheus Metrics

title

> Description: El título de la tarjeta.
>
> Source: frontend/src/components/molecules/Carousel.jsx, line 100

El título de la tarjeta.

(constant) uri :string

Description: La URI de conexión a la base de datos MongoDB.

Source: backend/src/server.js, line 63

La URI de conexión a la base de datos MongoDB.

Type:

- string

Methods

ActionCard(props) → {JSX.Element}

Description: Renderiza una tarjeta interactiva que redirige al usuario al hacer clic en un botón.

Source: frontend/src/components/molecules/ActionCard.jsx, line 16

Parameters:

props	*object*	Las propiedades del componente.

Properties

title	*string*			El título de la tarjeta.
description	*string*			La descripción de la tarjeta.
buttonText	*string*			El texto del botón.
navigateTo	*string*			La ruta a la que navegar al hacer clic en el botón.
variant	*string*	<optional>	'primary'	La variante de estilo de la tarjeta y el botón.
icon	*React.ReactNode*	<optional>		Un icono opcional para mostrar en la tarjeta.

Returns:

El componente de tarjeta de acción.

Type `JSX.Element`

ActivitySelector(props) → {JSX.Element}

Description: Un componente que permite a los usuarios seleccionar, añadir y eliminar actividades físicas.

Source: frontend/src/components/molecules/ActivitySelector.jsx, line 24

Parameters:

props	*object*	Las propiedades del componente.

Properties

actividades	*Array.<object>*	La lista de actividades ya registradas.
onChange	*function*	Función que se llama cuando la lista de actividades cambia.

Returns:

El componente selector de actividades.

Type `JSX.Element`

App() → {JSX.Element}

Description: Componente principal que renderiza la aplicación, gestiona el enrutamiento y los layouts.

Source: frontend/src/App.js, line 40

Returns:

El componente de la aplicación.

Type `JSX.Element`

Articulos() → {JSX.Element}

Description: Componente principal para mostrar artículos sobre salud mental

Source: frontend/src/pages/Articulos.jsx, line 13

Returns:

La página de artículos

Type `JSX.Element`

AuthButtons() → {JSX.Element}

Description: Renderiza los botones para navegar a las páginas de registro e inicio de sesión.

Source: frontend/src/components/molecules/AuthButtons.jsx, line 14

Returns:

El componente de botones de autenticación.

Type `JSX.Element`

`AuthLayout(props) → {JSX.Element}`

Description: Renderiza un layout centrado para los formularios de autenticación.

Source: frontend/src/components/layout/AuthLayout.jsx, line 12

Parameters:

`props`	*object*	Las propiedades del componente.

Properties

`children`	*React.ReactNode*	El contenido a renderizar dentro del layout (e.g., el formulario de login o registro).

Returns:

El componente de layout de autenticación.

Type `JSX.Element`

`Button(props) → {JSX.Element}`

Description: Renderiza un elemento de botón con estilos y funcionalidades personalizables.

Source: frontend/src/components/atoms/Button.jsx, line 12

Parameters:

`props`	*object*	Las propiedades del componente.

Properties

`children`	*React.ReactNode*			El contenido del botón.
`onClick`	*function*	<optional>		Función a ejecutar cuando se hace clic en el botón.
`variant`	*string*	<optional>	`'primary'`	La variante de estilo del botón ('primary',

				'secondary', 'outline').
`size`	*string*	<optional>	`'medium'`	El tamaño del botón ('small', 'medium', 'large').
`fullWidth`	*boolean*	<optional>	`false`	Si es `true`, el botón ocupa todo el ancho disponible.
`disabled`	*boolean*	<optional>	`false`	Si es `true`, el botón está deshabilitado.
`loading`	*boolean*	<optional>	`false`	Si es `true`, muestra un indicador de carga.
`type`	*string*	<optional>	`'button'`	El tipo de botón ('button', 'submit', 'reset').

Returns:

El componente de botón.

Type `JSX.Element`

`Card(props) → {JSX.Element}`

Description: Renderiza un componente de tarjeta con un título y una descripción.

Source: frontend/src/components/atoms/Card.jsx, line 12

Parameters:

`props`	*object*	Las propiedades del componente.

Properties

`title`	*string*	El título que se mostrará en la tarjeta.
`description`	*string*	La descripción que se mostrará en la tarjeta.

Returns:

El componente de tarjeta.

Type `JSX.Element`

Carousel(props) → {JSX.Element}

Description: Renderiza un carrusel de tarjetas con controles de navegación.

Source: frontend/src/components/molecules/Carousel.jsx, line 14

Parameters:

`props`	*object*	Las propiedades del componente.

Properties

`items`	*Array.<object>*	Una lista de objetos, cada uno con `title` y `description` para una tarjeta.

Returns:

El componente de carrusel.

Type `JSX.Element`

Checkbox(props) → {JSX.Element}

Description: Renderiza un input de tipo checkbox con una etiqueta.

Source: frontend/src/components/atoms/Checkbox.jsx, line 12

Parameters:

`props`	*object*	Las propiedades del componente.

Properties

`label`	*string*			La etiqueta que se mostrará junto al checkbox.
`checked`	*boolean*			Si es `true`, el checkbox estará marcado.
`onChange`	*function*			Función que se ejecuta cuando cambia el estado del checkbox.
`name`	*string*	<optional>		El nombre del input del checkbox.
`disabled`	*boolean*	<optional>	`false`	Si es `true`, el checkbox estará deshabilitado.

Returns:

El componente de checkbox.

Type `JSX.Element`

`CircularProgress(props) → {JSX.Element}`

Description: Renderiza un indicador de progreso circular.

Source: frontend/src/components/atoms/CircularProgress.jsx, line 12

Parameters:

props	*object*	Las propiedades del componente.		

Properties

percentage	*number*			El porcentaje de progreso a mostrar (0-100).
size	*number*	<optional>	120	El tamaño (ancho y alto) del componente.
strokeWidth	*number*	<optional>	10	El grosor del trazo del círculo.
color	*string*	<optional>	'#4f46e5'	El color del trazo de progreso.
label	*string*	<optional>	''	Una etiqueta opcional para mostrar debajo del porcentaje.

Returns:

El componente de progreso circular.

Type `JSX.Element`

`CognitionSelector(props) → {JSX.Element}`

Description: Un componente que permite a los usuarios seleccionar aspectos cognitivos y su intensidad.

Source: frontend/src/components/molecules/CognitionSelector.jsx, line 26

Parameters:

props	*object*	Las propiedades del componente.

Properties

`cognicion`	*Array.<object>*	La lista de aspectos cognitivos seleccionados.
`onChange`	*function*	Función que se llama cuando la selección cambia.

Returns:

El componente selector de cognición.

Type `JSX.Element`

`Diario() → {JSX.Element}`

Description: Componente principal de la página del diario. Gestiona el estado y la lógica para las entradas del diario.

Source: frontend/src/pages/Diario.jsx, line 22

Returns:

La página del diario.

Type `JSX.Element`

`DiaryBanner(props) → {JSX.Element}`

Description: Renderiza un banner promocional para la sección del diario.

Source: frontend/src/components/molecules/DiaryBanner.jsx, line 16

Parameters:

`props`	*object*	Las propiedades del componente.

Properties

`title`	*string*	El título del banner.
`description`	*string*	La descripción del banner.
`buttonText`	*string*	El texto del botón.
`navigateTo`	*string*	La ruta a la que navegar al hacer clic en el botón.

Returns:

El componente de banner del diario.

Type `JSX.Element`

`DiaryEditor(props) → {JSX.Element}`

Description: Renderiza un formulario para crear o editar una entrada del diario.

Source: frontend/src/components/molecules/DiaryEditor.jsx, line 20

Parameters:

`props`	*object*	Las propiedades del componente.

Properties

`onSave`	*function*			Función que se llama al guardar la entrada.
`onCancel`	*function*			Función que se llama al cancelar la edición.
`initialData`	*object*	<optional>	`null`	Los datos iniciales para editar una entrada existente.
`isLoading`	*boolean*	<optional>	`false`	Si es `true`, muestra un estado de carga.

Returns:

El componente del editor del diario.

Type `JSX.Element`

DiaryEntry(props) → {JSX.Element}

Description: Renderiza una única entrada del diario.

Source: frontend/src/components/molecules/DiaryEntry.jsx, line 14

Parameters:

`props`	*object*	Las propiedades del componente.

Properties

`entry`	*object*	El objeto de la entrada del diario.
`onEdit`	*function*	Función que se llama al hacer clic en editar.
`onDelete`	*function*	Función que se llama al hacer clic en eliminar.

`onShare`	*function*			Función que se llama al hacer clic en compartir.
`isOwner`	*boolean*	<optional>	`true`	Si es `true`, muestra los controles de propietario (editar, eliminar).

Returns:

El componente de la entrada del diario.

Type `JSX.Element`

`EmergencyButton(props) → {JSX.Element}`

Description: Componente que representa un botón de emergencia flotante.

Source: frontend/src/components/atoms/EmergencyButton.jsx, line 9

Parameters:

`props`	*object*	Propiedades del componente.

Properties

`onClick`	*function*	Función a ejecutar al hacer clic en el botón.

Returns:

El componente del botón de emergencia.

Type `JSX.Element`

`EmergencyModal(props) → {JSX.Element}`

Description: Componente que renderiza un modal con pestañas para información de emergencia, contactos y opciones de llamada.

Source: frontend/src/components/organisms/EmergencyModal.jsx, line 11

Parameters:

`props`	*object*	Propiedades del componente.

Properties

`onClose`	*function*	Función para cerrar el modal.

Returns:

El componente del modal de emergencia.

Type `JSX.Element`

`EmotionSelector(props) → {JSX.Element}`

Description: Un componente que permite a los usuarios seleccionar emociones, su intensidad y añadir un comentario.

Source: frontend/src/components/molecules/EmotionSelector.jsx, line 26

Parameters:

`props`	*object*	Las propiedades del componente.

Properties

`emociones`	*Array.<object>*	La lista de emociones seleccionadas.
`onChange`	*function*	Función que se llama cuando la selección de emociones cambia.
`comentario`	*string*	El comentario adicional del usuario.
`onComentarioChange`	*function*	Función que se llama cuando el comentario cambia.

Returns:

El componente selector de emociones.

Type `JSX.Element`

`EmotionStats(props) → {JSX.Element}`

Description: Renderiza una cuadrícula de estadísticas de emociones.

Source: frontend/src/components/molecules/EmotionStats.jsx, line 14

Parameters:

`props`	*object*	Las propiedades del componente.

Properties

`stats`	*Array.<object>*	Una lista de objetos de estadísticas, cada uno con `emotion` y `percentage`.

Returns:

El componente de estadísticas de emociones.

Type `JSX.Element`

`Footer() → {JSX.Element}`

Description: Renderiza el pie de página de la aplicación.

Source: frontend/src/components/molecules/Footer.jsx, line 12

Returns:

El componente del pie de página.

Type `JSX.Element`

`Heading(props) → {JSX.Element}`

Description: Renderiza un componente de encabezado con un nivel y clases personalizables.

Source: frontend/src/components/atoms/Heading.jsx, line 12

Parameters:

`props`	*object*	Las propiedades del componente.		

Properties

`level`	*number*	<optional>	`1`	El nivel del encabezado (1 para `<h1>`, 2 para `<h2>`, etc.).
`children`	*React.ReactNode*			El contenido del encabezado.
`className`	*string*	<optional>	`''`	Clases CSS adicionales para aplicar al encabezado.

Returns:

El componente de encabezado.

Type `JSX.Element`

`HeroSection(props) → {JSX.Element}`

Description: Renderiza una sección de héroe con un título y una descripción.

Source: frontend/src/components/molecules/HeroSection.jsx, line 16

Parameters:

`props`	*object*	Las propiedades del componente.

Properties

`title`	*string*	El título principal de la sección.
`description`	*string*	La descripción o subtítulo de la sección.

Returns:

El componente de la sección de héroe.

Type `JSX.Element`

`Home() → {JSX.Element}`

Description: Componente principal de la página de inicio. Muestra un saludo personalizado y varios widgets de información.

Source: frontend/src/pages/Home.jsx, line 20

Returns:

La página de inicio.

Type `JSX.Element`

`Input(props) → {JSX.Element}`

Description: Renderiza un campo de entrada con varias funcionalidades.

Source: frontend/src/components/atoms/Input.jsx, line 12

Parameters:

`props`	*object*	Las propiedades del componente.

Properties

`type`	*string*	<optional>	`'text'`	El tipo de input (e.g., 'text', 'password', 'email').
`value`	*string*			El valor del input.
`onChange`	*function*			Función que se ejecuta cuando cambia el

				valor del input.
`onBlur`	*function*	<optional>		Función que se ejecuta cuando el input pierde el foco.
`placeholder`	*string*	<optional>		El texto de marcador de posición del input.
`name`	*string*	<optional>		El nombre del input.
`disabled`	*boolean*	<optional>	`false`	Si es `true`, el input está deshabilitado.
`className`	*string*	<optional>	`''`	Clases CSS adicionales para el contenedor del input.
`error`	*string*	<optional>		Un mensaje de error para mostrar debajo del input.
`label`	*string*	<optional>		La etiqueta para mostrar encima del input.
`icon`	*React.ReactNode*	<optional>		Un icono para mostrar dentro del input.

Returns:

El componente de input.

Type `JSX.Element`

`Landing() → {JSX.Element}`

Description: Componente principal de la página de aterrizaje.

Source: frontend/src/pages/Landing.jsx, line 13

Returns:

La página de aterrizaje.

Type `JSX.Element`

`LandingHero() → {JSX.Element}`

Description: Renderiza la sección principal de la página de aterrizaje.

Source: frontend/src/components/organisms/LandingHero.jsx, line 14

Returns:

El componente Hero de la página de aterrizaje.

Type `JSX.Element`

`Login() → {JSX.Element}`

Description: Componente que renderiza el formulario de inicio de sesión y maneja la lógica de autenticación.

Source: frontend/src/pages/Login.js, line 18

Returns:

La página de inicio de sesión.

Type `JSX.Element`

`Logo() → {JSX.Element}`

Description: Renderiza el logo de la aplicación como un enlace a la página principal.

Source: frontend/src/components/atoms/Logo.jsx, line 14

Returns:

El componente del logo.

Type `JSX.Element`

`MainLayout(props) → {JSX.Element}`

Description: Renderiza el layout principal para las páginas de la aplicación.

Source: frontend/src/components/layout/MainLayout.jsx, line 16

Parameters:

props	*object*	Las propiedades del componente. **Properties** **children** — *React.ReactNode* — El contenido principal de la página a renderizar.

Returns:

El componente del layout principal.

Type `JSX.Element`

`Navbar() → {JSX.Element}`

Description: Renderiza la barra de navegación. Muestra diferentes enlaces si el usuario está autenticado o no.

Source: frontend/src/components/molecules/Navbar.jsx, line 14

Returns:

El componente de la barra de navegación.

Type `JSX.Element`

`ProtectedRoute(props) → {JSX.Element}`

Description: Redirige a los usuarios no autenticados a la página de login.

Source: frontend/src/App.js, line 48

Parameters:

`props`	*object*	Las propiedades del componente.

Properties

`children`	*React.ReactNode*	Los componentes hijos que se renderizarán si el usuario está autenticado.

Returns:

Los componentes hijos o una redirección a /login.

Type `JSX.Element`

`Register() → {JSX.Element}`

Description: Componente que renderiza el formulario de registro y maneja la lógica de creación de cuenta.

Source: frontend/src/pages/Register.js, line 16

Returns:

La página de registro.

Type `JSX.Element`

`Seguimiento() → {JSX.Element}`

Description: Componente principal de la página de seguimiento. Gestiona el estado del formulario de varios pasos y el envío de datos.

Source: frontend/src/pages/Seguimiento.jsx, line 28

Returns:

La página de seguimiento.

Type `JSX.Element`

`Slider(props) → {JSX.Element}`

Description: Renderiza un control deslizante (slider) para seleccionar un valor dentro de un rango.

Source: frontend/src/components/atoms/Slider.jsx, line 12

Parameters:

`props`	*object*	Las propiedades del componente.

Properties

`label`	*string*			La etiqueta que se mostrará encima del slider.
`value`	*number*			El valor actual del slider.
`onChange`	*function*			Función que se ejecuta cuando cambia el valor del slider.
`min`	*number*	<optional>	`1`	El valor mínimo del slider.
`max`	*number*	<optional>	`10`	El valor máximo del slider.
`step`	*number*	<optional>	`1`	El incremento entre los valores del slider.
`showValue`	*boolean*	<optional>	`true`	Si es `true`, muestra el valor actual junto a la etiqueta.

Returns:

El componente de slider.

Type `JSX.Element`

`Text(props) → {JSX.Element}`

Description: Renderiza un componente de texto con variantes y clases personalizables.

Source: frontend/src/components/atoms/Text.jsx, line 12

Parameters:

`props`	*object*	Las propiedades del componente.

Properties

`children`	*React.ReactNode*			El contenido del texto.
`variant`	*string*	<optional>	`'body'`	La variante de estilo del texto ('body', 'subtitle', 'caption',

				'small').
`className`	*string*	<optional>	`''`	Clases CSS adicionales para aplicar al texto.

Returns:

El componente de texto.

Type `JSX.Element`

`Textarea(props) → {JSX.Element}`

Description: Renderiza un campo de área de texto con funcionalidades adicionales.

Source: frontend/src/components/atoms/Textarea.jsx, line 12

Parameters:

`props`	*object*	Las propiedades del componente.		

Properties

`value`	*string*			El valor del área de texto.
`onChange`	*function*			Función que se ejecuta cuando cambia el valor.
`onBlur`	*function*	<optional>		Función que se ejecuta cuando el área de texto pierde el foco.
`placeholder`	*string*	<optional>		El texto de marcador de posición.
`name`	*string*	<optional>		El nombre del área de texto.
`disabled`	*boolean*	<optional>	`false`	Si es `true`, el área de texto está deshabilitada.
`rows`	*number*	<optional>	`4`	El número de filas visibles del área de texto.
`maxLength`	*number*	<optional>		El número máximo de caracteres permitidos.
`className`	*string*	<optional>	`''`	Clases CSS adicionales para el contenedor.
`error`	*string*	<optional>		Un mensaje de error

		para mostrar debajo.

Returns:

El componente de área de texto.

Type `JSX.Element`

(async) accessWithPassword(id, password) → {Promise.<object>}

Description: Accede a una entrada de diario protegida mediante una contraseña.

Source: frontend/src/service/diario.service.js, line 73

Parameters:

id	*string*	El ID de la entrada a la que se quiere acceder.
password	*string*	La contraseña para acceder a la entrada.

Returns:

La entrada del diario si la contraseña es correcta.

Type `Promise.<object>`

actualizarEntradaDiario(req, res) → {Promise.<void>}

Description: Actualiza una entrada del diario existente.

Source: backend/src/controllers/diario.controller.js, line 135

Parameters:

req	*object*	Objeto de petición de Express.
res	*object*	Objeto de respuesta de Express.

Returns:

Type `Promise.<void>`

(async) apiFetch(endpoint, options OPT) → {Promise.<Response>}

Description: Una función de ayuda para realizar peticiones `fetch` a la API, añadiendo automáticamente el token de autenticación.

Source: frontend/src/config/api.js, line 32

Parameters:

endpoint	*string*	El endpoint de la API al que se hará la

				petición.
options	*object*	<optional>	{}	Opciones adicionales para la petición fetch.

Returns:

La respuesta de la petición `fetch`.

Type `Promise.<Response>`

authMiddleware(req, res, next)

Description: El token debe ser enviado en el header `Authorization` como `Bearer <token>`.

Source: backend/src/middleware/authMiddleware.js, line 8

Parameters:

req	*object*	Objeto de petición de Express.
res	*object*	Objeto de respuesta de Express.
next	*function*	Función para pasar al siguiente middleware.

buscarArticulos(termino) → {Promise.<Array>}

Description: Busca artículos por término

Source: frontend/src/service/articlesService.js, line 391

Parameters:

termino	*string*	Término de búsqueda

Returns:

Artículos filtrados

Type `Promise.<Array>`

(async) cargarDatos()

Description: Carga los artículos reales de PubMed y categorías disponibles

Source: frontend/src/pages/Articulos.jsx, line 44

clasificarCategoria(texto) → {string}

Description: Clasifica un artículo por su contenido

Source: frontend/src/service/medicalAPIs.js, line 246

Parameters:

`texto`	*string*	Texto del artículo

Returns:

Categoría identificada

Type `string`

`closeShareModal()`

Description: Cierra el modal de compartir.

Source: frontend/src/pages/Diario.jsx, line 160

`compararPassword(candidatePassword) → {Promise.<boolean>}`

Description: Compara una contraseña candidata con la contraseña hasheada de la entrada del diario.

Source: backend/src/models/diario_mongoose.js, line 63

Parameters:

`candidatePassword`	*string*	La contraseña a comparar.

Returns:

- `true` si las contraseñas coinciden, `false` en caso contrario.

Type `Promise.<boolean>`

`componentDidCatch(error, errorInfo)`

Description: Captura información sobre el error.

Source: frontend/src/components/organisms/ErrorBoundary.jsx, line 40

Parameters:

`error`	*Error*	El error que fue lanzado.
`errorInfo`	*object*	Un objeto con una clave `componentStack` que contiene información sobre qué componente lanzó el error.

`(async) connectDBAndStartServer()`

Description: Conecta a la base de datos MongoDB y, si tiene éxito, inicia el servidor Express.

Source: backend/src/server.js, line 74

`copyShareLink()`

Description: Copia el enlace para compartir al portapapeles.

Source: frontend/src/pages/Diario.jsx, line 150

crearEntradaDiario(req, res) → {Promise.<void>}

Description: Crea una nueva entrada en el diario para el usuario autenticado.

Source: backend/src/controllers/diario.controller.js, line 8

Parameters:

req	*object*	Objeto de petición de Express.
res	*object*	Objeto de respuesta de Express.

Returns:

Type `Promise.<void>`

(async) create(contactData) → {Promise.<object>}

Description: Crea un nuevo contacto de emergencia.

Source: frontend/src/service/contactoEmergencia.service.js, line 24

Parameters:

contactData	*object*	Datos del contacto (nombre, telefono, email, relacion).

Returns:

El contacto creado.

Type `Promise.<object>`

(async) create(diarioData) → {Promise.<object>}

Description: Crea una nueva entrada en el diario.

Source: frontend/src/service/diario.service.js, line 13

Parameters:

diarioData	*object*	Los datos de la entrada a crear (título, cuerpo, etc.).

Returns:

La nueva entrada creada.

Type `Promise.<object>`

createContacto(req, res) → {Promise.<void>}

Description: Crea un nuevo contacto de emergencia para el usuario autenticado.

Source: backend/src/controllers/contactoEmergencia.controller.js, line 12

Parameters:

req	*object*	Objeto de petición de Express.
res	*object*	Objeto de respuesta de Express.

Returns:

Type `Promise.<void>`

`createRegistro(req, res)`

Description: Crea un nuevo registro diario del usuario

Source: backend/src/controllers/registro.controller.js, line 3

Parameters:

req	*object*	Request object
res	*object*	Response object

`(async) delete(id) → {Promise.<object>}`

Description: Elimina una entrada del diario.

Source: frontend/src/service/diario.service.js, line 61

Parameters:

id	*string*	El ID de la entrada a eliminar.

Returns:

Un mensaje de confirmación.

Type `Promise.<object>`

`deleteContacto(req, res) → {Promise.<void>}`

Description: Elimina un contacto de emergencia.

Source: backend/src/controllers/contactoEmergencia.controller.js, line 103

Parameters:

req	*object*	Objeto de petición de Express.
res	*object*	Objeto de respuesta de Express.

Returns:

Type `Promise.<void>`

dismiss(toastId)

Description: Oculta un toast específico por su ID.

Source: frontend/src/hooks/useToast.js, line 75

Parameters:

`toastId`	*string*	El ID del toast a ocultar.

eliminarEntradaDiario(req, res) → {Promise.<void>}

Description: Elimina una entrada del diario.

Source: backend/src/controllers/diario.controller.js, line 174

Parameters:

`req`	*object*	Objeto de petición de Express.
`res`	*object*	Objeto de respuesta de Express.

Returns:

Type `Promise.<void>`

error(message, options OPT)

Description: Muestra un toast de error.

Source: frontend/src/hooks/useToast.js, line 34

Parameters:

`message`	*string*		El mensaje a mostrar.
`options`	*object*	<optional>	Opciones adicionales para el toast.

filtrarArticulos() → {Array}

Description: Filtra los artículos según búsqueda y categoría

Source: frontend/src/pages/Articulos.jsx, line 70

Returns:

Artículos filtrados

Type `Array`

formatDate(date) → {string}

Description: Formatea una fecha a un formato largo y legible.

Source: frontend/src/utils/formatters.js, line 6

Parameters:

date	*string* | *Date*	La fecha a formatear.

Returns:

La fecha formateada (e.g., "2 de diciembre de 2025, 14:30").

Type `string`

formatDateShort(date) → {string}

Description: Formatea una fecha a un formato corto (DD/MM/YYYY).

Source: frontend/src/utils/formatters.js, line 31

Parameters:

date	*string* | *Date*	La fecha a formatear.

Returns:

La fecha formateada en formato corto.

Type `string`

(async) getAll() → {Promise.<Array.<object>>}

Description: Obtiene todos los contactos de emergencia del usuario autenticado.

Source: frontend/src/service/contactoEmergencia.service.js, line 13

Returns:

Una lista de los contactos de emergencia.

Type `Promise.<Array.<object>>`

(async) getAll() → {Promise.<Array.<object>>}

Description: Obtiene todas las entradas del diario del usuario autenticado.

Source: frontend/src/service/diario.service.js, line 25

Returns:

Una lista de las entradas del diario.

Type `Promise.<Array.<object>>`

(async) getById(id) → {Promise.<object>}

Description: Obtiene una entrada del diario por su ID.

Source: frontend/src/service/diario.service.js, line 36

Parameters:

id	*string*	El ID de la entrada a obtener.

Returns:

La entrada del diario solicitada.

Type `Promise.<object>`

getContactoById(req, res) → {Promise.<void>}

Description: Obtiene un contacto de emergencia específico por su ID.

Source: backend/src/controllers/contactoEmergencia.controller.js, line 58

Parameters:

req	*object*	Objeto de petición de Express.
res	*object*	Objeto de respuesta de Express.

Returns:

Type `Promise.<void>`

getContactos(req, res) → {Promise.<void>}

Description: Obtiene todos los contactos de emergencia del usuario autenticado.

Source: backend/src/controllers/contactoEmergencia.controller.js, line 42

Parameters:

req	*object*	Objeto de petición de Express.
res	*object*	Objeto de respuesta de Express.

Returns:

Type `Promise.<void>`

(static) getDerivedStateFromError(error) → {object}

Description: Actualiza el estado para que el siguiente renderizado muestre la UI de fallback.

Source: frontend/src/components/organisms/ErrorBoundary.jsx, line 29

Parameters:

`error`	*Error*	El error que fue lanzado.

Returns:

Un objeto de estado que indica que ha ocurrido un error.

Type `object`

`getIntensidad(aspecto) → {number}`

Description: Obtiene la intensidad de un aspecto cognitivo seleccionado.

Source: frontend/src/components/molecules/CognitionSelector.jsx, line 69

Parameters:

`aspecto`	*string*	El aspecto cognitivo.

Returns:

La intensidad del aspecto, o 5 por defecto.

Type `number`

`getIntensidad(emocion) → {number}`

Description: Obtiene la intensidad de una emoción seleccionada.

Source: frontend/src/components/molecules/EmotionSelector.jsx, line 71

Parameters:

`emocion`	*string*	La emoción.

Returns:

La intensidad de la emoción, o 5 por defecto.

Type `number`

`(async) getMetrics() → {Promise.<string>}`

Description: Obtener métricas en formato Prometheus

Source: backend/src/services/observability.service.js, line 240

Returns:

Métricas formateadas

Type `Promise.<string>`

`getPreviewText() → {string}`

Description: Devuelve el texto completo o una vista previa del cuerpo de la entrada.

Source: frontend/src/components/molecules/DiaryEntry.jsx, line 44

Returns:

El texto a mostrar.

Type `string`

getProfile(req, res) → {Promise.<void>}

Description: Obtiene el perfil del usuario autenticado.

Source: backend/src/controllers/auth.controller.js, line 182

Parameters:

req	*object*	Objeto de petición de Express, con la información del usuario en `req.user`.
res	*object*	Objeto de respuesta de Express.

Returns:

Type `Promise.<void>`

getRegistroByFecha(req, res)

Description: Obtiene registros de una fecha específica

Source: backend/src/controllers/registro.controller.js, line 103

Parameters:

req	*object*	Request object
res	*object*	Response object

getRegistroById(req, res)

Description: Obtiene un registro específico por su ID

Source: backend/src/controllers/registro.controller.js, line 69

Parameters:

req	*object*	Request object
res	*object*	Response object

getRegistros(req, res)

Description: Obtiene todos los registros del usuario autenticado

Source: backend/src/controllers/registro.controller.js, line 40

Parameters:

req	*object*	Request object
res	*object*	Response object

getRegistrosByRango(req, res)

Description: Obtiene registros en un rango de fechas

Source: backend/src/controllers/registro.controller.js, line 148

Parameters:

req	*object*	Request object
res	*object*	Response object

getStepLabel(step) → {string}

Description: Obtiene la etiqueta para un paso específico del formulario.

Source: frontend/src/pages/Seguimiento.jsx, line 123

Parameters:

step	*number*	El número del paso.

Returns:

La etiqueta del paso.

Type `string`

goToNext()

Description: Navega a la siguiente diapositiva en el carrusel.

Source: frontend/src/components/molecules/Carousel.jsx, line 35

goToPrevious()

Description: Navega a la diapositiva anterior en el carrusel.

Source: frontend/src/components/molecules/Carousel.jsx, line 25

handleAddActividad()

Description: Añade la nueva actividad a la lista de actividades.

Source: frontend/src/components/molecules/ActivitySelector.jsx, line 39

handleAddContact(e)

Description: Añade un nuevo contacto de emergencia.

Source: frontend/src/components/organisms/EmergencyModal.jsx, line 129

Parameters:

e	*Event*	El evento de submit.

handleCall(number)

Description: Inicia una llamada telefónica si el dispositivo es móvil, de lo contrario muestra una alerta.

Source: frontend/src/components/organisms/EmergencyModal.jsx, line 63

Parameters:

number	*string*	El número de teléfono a llamar.

handleCancelAdd()

Description: Cancela la adición de un nuevo contacto.

Source: frontend/src/components/organisms/EmergencyModal.jsx, line 156

handleCancelEditor()

Description: Cancela la edición y cierra el editor.

Source: frontend/src/pages/Diario.jsx, line 132

handleChange(e)

Description: Actualiza el estado del formulario cuando cambian los campos de entrada.

Source: frontend/src/components/molecules/DiaryEditor.jsx, line 39

Parameters:

e	*React.ChangeEvent.<(HTMLInputElement|HTMLTextAreaElement)>*	El evento de cambio.

(async) handleCreateEntry(entryData)

Description: Maneja la creación de una nueva entrada del diario.

Source: frontend/src/pages/Diario.jsx, line 58

Parameters:

`entryData`	*object*	Los datos de la nueva entrada.

(async) handleDeleteEntry(entryId)

Description: Maneja la eliminación de una entrada.

Source: frontend/src/pages/Diario.jsx, line 99

Parameters:

`entryId`	*string*	El ID de la entrada a eliminar.

handleEditEntry(entry)

Description: Prepara el editor para modificar una entrada existente.

Source: frontend/src/pages/Diario.jsx, line 122

Parameters:

`entry`	*object*	La entrada a editar.

handleEmocionToggle(emocion)

Description: Añade o elimina una emoción de la selección.

Source: frontend/src/components/molecules/EmotionSelector.jsx, line 37

Parameters:

`emocion`	*string*	La emoción a alternar.

handleImageError(e, articuloId)

Description: Maneja errores de carga de imágenes usando una imagen por defecto

Source: frontend/src/pages/Articulos.jsx, line 31

Parameters:

`e`	*Event*	Evento de error
`articuloId`	*number*	ID del artículo

handleInputChange(e)

Description: Maneja los cambios en los inputs del formulario.

Source: frontend/src/components/organisms/EmergencyModal.jsx, line 116

Parameters:

e	*Event*	El evento de cambio.

handleIntensidadChange(aspecto, intensidad)

Description: Cambia la intensidad de un aspecto cognitivo seleccionado.

Source: frontend/src/components/molecules/CognitionSelector.jsx, line 49

Parameters:

aspecto	*string*	El aspecto cognitivo a modificar.
intensidad	*number*	El nuevo valor de intensidad.

handleIntensidadChange(emocion, intensidad)

Description: Cambia la intensidad de una emoción seleccionada.

Source: frontend/src/components/molecules/EmotionSelector.jsx, line 51

Parameters:

emocion	*string*	La emoción a modificar.
intensidad	*number*	El nuevo valor de intensidad.

handleLogin()

Description: Navega a la página de inicio de sesión.

Source: frontend/src/components/molecules/AuthButtons.jsx, line 30

(async) handleLogin(e)

Description: Envía las credenciales del usuario al backend para la autenticación.

Source: frontend/src/pages/Login.js, line 38

Parameters:

e	*React.FormEvent.<HTMLFormElement>*	El evento de envío del formulario.

handleLogout()

Description: Cierra la sesión del usuario y lo redirige a la página de inicio.

Source: frontend/src/components/molecules/Navbar.jsx, line 23

handleOverlayClick(e)

Description: Cierra el modal si se hace clic en el overlay.

Source: frontend/src/components/organisms/EmergencyModal.jsx, line 105

Parameters:

e	*Event*	El evento de clic.

handleRegister()

Description: Navega a la página de registro.

Source: frontend/src/components/molecules/AuthButtons.jsx, line 22

(async) handleRegister(e)

Description: Valida los datos del formulario y los envía al backend para crear un nuevo usuario.

Source: frontend/src/pages/Register.js, line 36

Parameters:

e	*React.FormEvent.<HTMLFormElement>*	El evento de envío del formulario.

handleReload()

Description: Recarga la página para intentar resolver el error.

Source: frontend/src/components/organisms/ErrorBoundary.jsx, line 51

handleRemoveActividad(index)

Description: Elimina una actividad de la lista por su índice.

Source: frontend/src/components/molecules/ActivitySelector.jsx, line 50

Parameters:

index	*number*	El índice de la actividad a eliminar.

handleSendEmail(contactoId, email)

Description: Envía un email de emergencia a través del backend.

Source: frontend/src/components/organisms/EmergencyModal.jsx, line 76

Parameters:

`contactoId`	*string*	El ID del contacto de emergencia.
`email`	*string*	La dirección de correo electrónico del destinatario (para mostrar en mensajes).

handleSendSms(phone)

Description: Inicia el envío de un SMS si el dispositivo es móvil, de lo contrario muestra una alerta.

Source: frontend/src/components/organisms/EmergencyModal.jsx, line 92

Parameters:

`phone`	*string*	El número de teléfono para enviar el SMS.

handleShareEntry(entry)

Description: Abre el modal para compartir una entrada.

Source: frontend/src/pages/Diario.jsx, line 141

Parameters:

`entry`	*object*	La entrada a compartir.

handleSubmit(e)

Description: Valida y envía los datos del formulario al guardar.

Source: frontend/src/components/molecules/DiaryEditor.jsx, line 49

Parameters:

`e`	*React.FormEvent.<HTMLFormElement>*	El evento de envío del formulario.

(async) handleSubmit()

Description: Envía los datos del formulario de seguimiento al backend.

Source: frontend/src/pages/Seguimiento.jsx, line 82

handleToggle(aspecto)

Description: Añade o elimina un aspecto cognitivo de la selección.

Source: frontend/src/components/molecules/CognitionSelector.jsx, line 35

Parameters:

aspecto	*string*		El aspecto cognitivo a alternar.

handleToggleExpand()

Description: Expande o contrae el cuerpo de la entrada del diario.

Source: frontend/src/components/molecules/DiaryEntry.jsx, line 36

(async) handleUpdateEntry(entryData)

Description: Maneja la actualización de una entrada existente.

Source: frontend/src/pages/Diario.jsx, line 78

Parameters:

entryData	*object*		Los nuevos datos para la entrada.

healthCheckController(req, res)

Description: Controlador que verifica si la API está disponible

Source: backend/src/routes/health.routes.js, line 4

Parameters:

req	*object*		Objeto de petición
res	*object*		Objeto de respuesta

info(message, options OPT)

Description: Muestra un toast de información.

Source: frontend/src/hooks/useToast.js, line 48

Parameters:

message	*string*		El mensaje a mostrar.
options	*object*	<optional>	Opciones adicionales para el toast.

instrumentMongoDB(schema)

Description: Instrumentación para MongoDB con Mongoose

Source: backend/src/services/observability.service.js, line 171

Parameters:

`schema`	*Object*	Schema de Mongoose

`isMobileDevice() → {boolean}`

Description: Detecta si el usuario está en un dispositivo móvil.

Source: frontend/src/components/organisms/EmergencyModal.jsx, line 54

Returns:

Type `boolean`

`isSelected(aspecto) → {boolean}`

Description: Comprueba si un aspecto cognitivo está seleccionado.

Source: frontend/src/components/molecules/CognitionSelector.jsx, line 61

Parameters:

`aspecto`	*string*	El aspecto cognitivo a comprobar.

Returns:

`true` si está seleccionado, `false` en caso contrario.

Type `boolean`

`isSelected(emocion) → {boolean}`

Description: Comprueba si una emoción está seleccionada.

Source: frontend/src/components/molecules/EmotionSelector.jsx, line 63

Parameters:

`emocion`	*string*	La emoción a comprobar.

Returns:

`true` si está seleccionada, `false` en caso contrario.

Type `boolean`

`loadContacts()`

Description: Carga los contactos de emergencia desde la API.

Source: frontend/src/components/organisms/EmergencyModal.jsx, line 31

(async) loadEntries()

Description: Carga todas las entradas del diario del usuario desde el servicio.

Source: frontend/src/pages/Diario.jsx, line 41

loading(message) → {string}

Description: Muestra un toast de carga que puede ser actualizado o descartado.

Source: frontend/src/hooks/useToast.js, line 63

Parameters:

message	*string*	El mensaje a mostrar.

Returns:

El ID del toast de carga.

Type `string`

logAuthAttempt(endpoint, result)

Description: Registrar intento de autenticación

Source: backend/src/services/observability.service.js, line 231

Parameters:

endpoint	*string*	Endpoint de autenticación
result	*string*	Resultado ('success' o 'failure')

loginUser(req, res) → {Promise.<void>}

Description: Autentica a un usuario y le proporciona un token JWT.

Source: backend/src/controllers/auth.controller.js, line 112

Parameters:

req	*object*	Objeto de petición de Express.
res	*object*	Objeto de respuesta de Express.

Returns:

Type `Promise.<void>`

metricsMiddleware(req, res, next)

Description: Middleware para registrar métricas HTTP

Source: backend/src/services/observability.service.js, line 118

Parameters:

req	*Object*	Objeto de request de Express
res	*Object*	Objeto de response de Express
next	*function*	Función next de Express

nextStep()

Description: Avanza al siguiente paso del formulario.

Source: frontend/src/pages/Seguimiento.jsx, line 111

(async) obtenerArticulos(filtros) → {Promise.<Object>}

Description: Obtiene artículos de PubMed API real

Source: frontend/src/service/articlesService.js, line 17

Parameters:

filtros	*Object*	Filtros opcionales

Returns:

Lista de artículos

Type `Promise.<Object>`

(async) obtenerArticulosDeMedicalAPIs() → {Promise.<Array>}

Description: Obtiene artículos de APIs médicas públicas

Source: frontend/src/service/medicalAPIs.js, line 61

Returns:

Artículos médicos

Type `Promise.<Array>`

(async) obtenerArticulosDeNewsAPI(query) → {Promise.<Array>}

Description: Obtiene artículos de NewsAPI (requiere API key)

Source: frontend/src/service/medicalAPIs.js, line 30

Parameters:

query	*string*	Término de búsqueda

Returns:

Artículos de NewsAPI

Type `Promise.<Array>`

(async) obtenerArticulosDePubMed(query) → {Promise.<Array>}

Description: Obtiene artículos de PubMed (API pública)

Source: frontend/src/service/medicalAPIs.js, line 6

Parameters:

query	*string*	Término de búsqueda

Returns:

Artículos del servidor PubMed

Type `Promise.<Array>`

obtenerArticulosLocales() → {Array}

Description: Obtiene artículos de una base de datos local simulada

Source: frontend/src/service/medicalAPIs.js, line 98

Returns:

Artículos locales

Type `Array`

obtenerArticulosPorCategoria(categoria) → {Promise.<Array>}

Description: Obtiene artículos de una categoría específica

Source: frontend/src/service/articlesService.js, line 406

Parameters:

categoria	*string*	Categoría a filtrar

Returns:

Artículos de la categoría

Type `Promise.<Array>`

obtenerCategorias() → {Array}

Description: Obtiene todas las categorías disponibles

Source: frontend/src/service/articlesService.js, line 382

Returns:

Lista de categorías

Type `Array`

obtenerEntradaDiarioPorId(req, res) → {Promise.<void>}

Description: Si la entrada es pública (con contraseña), se verifica la contraseña.

Source: backend/src/controllers/diario.controller.js, line 85

Parameters:

req	*object*	Objeto de petición de Express.
res	*object*	Objeto de respuesta de Express.

Returns:

Type `Promise.<void>`

obtenerEntradasDiario(req, res) → {Promise.<void>}

Description: Obtiene todas las entradas del diario del usuario autenticado.

Source: backend/src/controllers/diario.controller.js, line 59

Parameters:

req	*object*	Objeto de petición de Express.
res	*object*	Objeto de respuesta de Express.

Returns:

Type `Promise.<void>`

optionalAuthMiddleware(req, res, next)

Description: Si no se proporciona un token o es inválido, simplemente continúa sin un usuario autenticado.

Source: backend/src/middleware/authMiddleware.js, line 84

Parameters:

req	*object*	Objeto de petición de Express.
res	*object*	Objeto de respuesta de Express.
next	*function*	Función para pasar al siguiente middleware.

pre-save()

Description: Se ejecuta solo si la contraseña ha sido modificada o es nueva.

Source: backend/src/models/diario_mongoose.js, line 43

prevStep()

Description: Retrocede al paso anterior del formulario.

Source: frontend/src/pages/Seguimiento.jsx, line 117

procesarArticulosDeNewsAPI(articulos) → {Array}

Description: Formatea artículos de NewsAPI a nuestro esquema

Source: frontend/src/service/medicalAPIs.js, line 225

Parameters:

articulos	*Array*	Artículos crudos

Returns:

Artículos formateados

Type Array

procesarArticulosDePubMed(articulos) → {Array}

Description: Formatea artículos de PubMed a nuestro esquema

Source: frontend/src/service/medicalAPIs.js, line 204

Parameters:

articulos	*Array*	Artículos crudos

Returns:

Artículos formateados

Type Array

promise(promise, messages) → {Promise.<any>}

Description: Maneja una promesa y muestra toasts de carga, éxito o error automáticamente.

Source: frontend/src/hooks/useToast.js, line 84

Parameters:

promise	*Promise.<any>*	La promesa a manejar.
messages	*object*	Los mensajes para cada estado de la promesa.

Properties

`loading`	*string*	<optional>	Mensaje mientras la promesa está pendiente.
`success`	*string*	<optional>	Mensaje cuando la promesa se resuelve con éxito.
`error`	*string*	<optional>	Mensaje cuando la promesa es rechazada.

Returns:

La promesa original.

Type `Promise.<any>`

`registerUser(req, res) → {Promise.<void>}`

Description: Registra un nuevo usuario en la base de datos.

Source: backend/src/controllers/auth.controller.js, line 10

Parameters:

`req`	*object*	Objeto de petición de Express.
`res`	*object*	Objeto de respuesta de Express.

Returns:

Type `Promise.<void>`

`render() → {React.ReactNode}`

Description: Renderiza el componente. Muestra la UI de fallback si hay un error, o los componentes hijos si no lo hay.

Source: frontend/src/components/organisms/ErrorBoundary.jsx, line 59

Returns:

Type `React.ReactNode`

`request-interceptor()`

Description: Interceptor de peticiones de Axios que añade el token de autenticación a las cabeceras.

Source: frontend/src/config/api.service.js, line 29

`response-interceptor()`

Description: Interceptor de respuestas de Axios que maneja errores, especialmente los de autenticación (401).

Source: frontend/src/config/api.service.js, line 56

sendEmergencyEmail(req, res) → {Promise.<void>}

Description: Envía un email de emergencia a un contacto.

Source: backend/src/controllers/contactoEmergencia.controller.js, line 129

Parameters:

req	*object*	Objeto de petición de Express.
res	*object*	Objeto de respuesta de Express.

Returns:

Type `Promise.<void>`

(async) sendEmergencyEmail(contactoId) → {Promise.<object>}

Description: Envía un email de emergencia a un contacto.

Source: frontend/src/service/contactoEmergencia.service.js, line 36

Parameters:

contactoId	*string*	ID del contacto de emergencia.

Returns:

Respuesta del servidor.

Type `Promise.<object>`

success(message, options OPT)

Description: Muestra un toast de éxito.

Source: frontend/src/hooks/useToast.js, line 20

Parameters:

message	*string*		El mensaje a mostrar.
options	*object*	<optional>	Opciones adicionales para el toast.

timeAgo(date) → {string}

Description: Calcula y formatea el tiempo transcurrido desde una fecha dada hasta ahora.

Source: frontend/src/utils/formatters.js, line 51

Parameters:

`date`	*string* | *Date*	La fecha desde la que calcular el tiempo transcurrido.

Returns:

Una cadena que representa el tiempo transcurrido (e.g., "Hace 2 horas").

Type `string`

`togglePasswordField()`

Description: Muestra u oculta el campo de la contraseña.

Source: frontend/src/components/molecules/DiaryEditor.jsx, line 82

`(async) update(id, diarioData) → {Promise.<object>}`

Description: Actualiza una entrada del diario existente.

Source: frontend/src/service/diario.service.js, line 48

Parameters:

`id`	*string*	El ID de la entrada a actualizar.
`diarioData`	*object*	Los nuevos datos para la entrada.

Returns:

La entrada actualizada.

Type `Promise.<object>`

`updateContacto(req, res) → {Promise.<void>}`

Description: Actualiza un contacto de emergencia existente.

Source: backend/src/controllers/contactoEmergencia.controller.js, line 77

Parameters:

`req`	*object*	Objeto de petición de Express.
`res`	*object*	Objeto de respuesta de Express.

Returns:

Type `Promise.<void>`

`useAuthStore() → {object}`

Description: Hook de Zustand para acceder y manipular el estado de autenticación.

Source: frontend/src/store/authStore.js, line 10

Properties:

`user`	`object` | `null`	El objeto de usuario si está autenticado, o `null`.
`token`	`string`	El token JWT de autenticación.
`isAuthenticated`	`function`	Una función computada que devuelve `true` si el usuario está autenticado.
`setAuth`	`function`	Acción para guardar los datos del usuario y el token.
`logout`	`function`	Acción para limpiar los datos de autenticación.

Returns:

El store de autenticación con el estado y las acciones.

Type `object`

`useToast() → {object}`

Description: Un hook personalizado que abstrae la librería `react-hot-toast` para un uso simplificado.

Source: frontend/src/hooks/useToast.js, line 8

Properties:

`success`	`function`	Muestra un toast de éxito.
`error`	`function`	Muestra un toast de error.
`info`	`function`	Muestra un toast de información.
`loading`	`function`	Muestra un toast de carga.
`dismiss`	`function`	Oculta un toast específico.
`promise`	`function`	Muestra toasts automáticamente para el ciclo de vida de una promesa.

Returns:

Un objeto con funciones para mostrar diferentes tipos de toasts.

Type `object`

`validateRequiredEnvVars()`

Description: Valida que las variables de entorno críticas estén configuradas

Source: backend/src/server.js, line 14

Type Definitions

`Diario`

Source: backend/src/models/diario_mongoose.js, line 13

Properties:

`usuarioId`	`mongoose.Schema.Types.ObjectId`		ID del usuario propietario de la entrada.
`titulo`	`string`		Título de la entrada del diario.
`cuerpo`	`string`		Contenido de la entrada del diario.
`password`	`string`	<optional>	Contraseña opcional para proteger la entrada.
`createdAt`	`Date`		Fecha de creación de la entrada.
`updatedAt`	`Date`		Fecha de la última actualización de la entrada.

Type:

- `object`

Events

`SIGINT`

Description: Maneja el cierre de la aplicación (Ctrl+C) para cerrar la conexión a la base de datos de forma segura.

Source: backend/src/server.js, line 102

`unhandledRejection`

Description: Maneja promesas rechazadas no capturadas para evitar que el proceso se bloquee.

Source: backend/src/server.js, line 114

Documentation generated by JSDoc 4.0.5 on Fri Feb 13 2026 07:53:17 GMT+0000 (Coordinated Universal Time) using the docdash theme.